

Databases	3
Creating Databases	3
Dropping Databases	3
Using Databases	3
Multiple Insert	3
INSERT Challenges	3
MySQL Warnings	4
NULL and NOT NULL	4
Setting Default Values + Not null	5
Primary Keys	5
Table Constraints Exercise	6
CRUD=Create Read Update Delete	7
Preparing Data	7
SELECT	7
WHERE	7
Aliases	8
Updating Data	8
Update Challenges	8
DELETING DATA	8
Delete Challenges Solution	8
Spring Cleaning – The Annual Closet Inventory –CRUD EXERCISE	9
The World Of String Functions.....	10
Running SQL Files	10
Loading Book Data	10
CONCAT	12
SUBSTRING	12
REPLACE	13
REVERSE	13
CHAR LENGTH	14
Changing Case with UPPER and LOWER	14
Note about string functions	14
String Function Challenges	14
DISTINCT	15
Sorting Data with ORDER BY	15
LIMIT	16
Searches with LIKE - WILDCARD	17
LIKE Part 2: More Wildcards	17
Refining Selections Exercises	17
The Magic of Aggregate Functions	17
The Count Function	17
The Joys of Group By	18
MIN and MAX Basics.....	18
A Problem with Min and Max.....	18
Min and Max with Group By	19
The Sum Function	19
The Avg Function.....	19
Aggregate Functions Challenges	20
Data types.....	20
CHAR and VARCHAR.....	20
DECIMAL.....	20
FLOAT and DOUBLE	21
DATE data.....	21
Formatting Dates	21
Date Math	22
Working with TIMESTAMPS	22
Data Types Exercises	23

Date and Time function Documentation	24
The Power of Logical Operators	24
Not Equal.....	24
Not Like.....	24
Greater Than.....	24
Less Than	25
Logical AND	25
Logical OR	26
Between.....	26
In And Not In	27
Case Statements.....	27
Logical Operators Exercises.....	28
One to many	29
Working With Foreign Keys	29
Cross Joins.....	30
Inner Joins	30
SET @@sql_mode=";	30
Left Joins	31
Right Joins.....	31
Right and Left Joins FAQ	33
Joins Exercise.....	33
MANY To Many.....	34
TV Review	34
IG Clone	37
Users Schema	37
Photos Schema	37
Comments Schema.....	38
Likes Schema	38
Followers Schema	38
Hashtags Schema.....	38
Section 15: Working With Lots Of Instagram Data	38
-- 1. Finding 5 oldest users	38
-- 2. Most Popular Registration Date.....	39
-- 3. Identify Inactive Users (users with no photos)	39
-- 4. Identify most popular photo (and user who created it)	39
-- 5. Calculate average number of photos per user.....	39
-- 6. Find the five most popular hashtags	39
-- 7. Finding the bots - the users who have liked every single photo	39
Bulk Inserting 500 Fake Users Node.js + MySql (Python Faker).....	40
500 Users Exercises	41
Challenge 1. Find Earliest Date A User Joined	41
Challenge 2. Find Email Of The First (Earliest)User	41
Challenge 3. Users According To The Month They Joined	41
Challenge 4. Count Number of Users With Yahoo Emails	41
Challenge 5. Calculate Total Number of Users for Each Email Host	41
https://stackoverflow.com/questions/2628138/how-to-select-domain-name-from-email-address	41
MySQL Database Triggers	42
Listing Triggers	43
Removing Triggers.....	43
A WORD OF WARNING	43
Preventing Self-Follows	43
Logging Unfollows.....	43

DATABASES

CREATING DATABASES

Start the CLI: `mysql-ctl cli;`

List available databases: `SHOW databases;`

The general command for creating a database: `CREATE DATABASE database_name;`

A specific example: `CREATE DATABASE soap_store;`

DROPPING DATABASES

`DROP DATABASE database_name;` For Example: `DROP DATABASE hello_world_db;`

Remember to be careful with this command! Once you drop a database, it's gone!

USING DATABASES

1. `USE <database name>;`
2. -- example: `USE dog_walking_app;`
3. `SELECT database();`

MULTIPLE INSERT

1. `INSERT INTO table_name`
2. `(column_name, column_name)`
3. `VALUES (value, value),`
4. `(value, value),`
5. `(value, value);`

INSERT CHALLENGES

1. `CREATE TABLE people`
 2. `(`
 3. `first_name VARCHAR(20),`
 4. `last_name VARCHAR(20),`
 5. `age INT`
 6. `);`
-
1. `INSERT INTO people(first_name, last_name, age)`
 2. `VALUES ('Tina', 'Belcher', 13);`
-
1. `INSERT INTO people(age, last_name, first_name)`
 2. `VALUES (42, 'Belcher', 'Bob');`
-
1. `INSERT INTO people(first_name, last_name, age)`
 2. `VALUES('Linda', 'Belcher', 45)`
 3. `,('Phillip', 'Fronnd', 38)`
 4. `,('Calvin', 'Fischoeder', 70);`

`DROP TABLE people;`

```
SELECT * FROM people;
```

```
SHOW tables;
```

MYSQL WARNINGS

```
DESC cats;
```

Try Inserting a cat with a super long name:

1. **INSERT INTO** cats(name, age)
2. **VALUES**('This is some text blah blah blah blah text text text something about cats lalalalal meowwwwwwwwwww', 10);

Then view the warning: **SHOW WARNINGS;**

Try inserting a cat with incorrect data types:

```
INSERT INTO cats(name, age) VALUES('Lima',  
'dsfasdfdas');
```

Then view the warning: **SHOW WARNINGS;**

NULL AND NOT NULL

Try inserting a cat **without an age**:

```
INSERT INTO cats(name) VALUES('Alabama');
```

```
SELECT * FROM cats;
```

Try inserting a **nameless** and **ageless** cat:

```
INSERT INTO cats() VALUES();
```

Define a new cats2 table with **NOT NULL constraints**:

1. **CREATE TABLE** cats2
2. (
3. name **VARCHAR**(100) **NOT NULL**,
4. age **INT NOT NULL**
5.);

```
DESC cats2;
```

Now try inserting an **ageless** cat:

```
INSERT INTO cats2(name) VALUES('Texas');
```

View the new warnings:

```
SHOW WARNINGS;
```

```
SELECT * FROM cats2;
```

Do the same for a nameless cat:

```
INSERT INTO cats2 (age) VALUES (7);

SHOW WARNINGS;
```

SETTING DEFAULT VALUES + NOT NULL

Define a table with a **DEFAULT** name specified:

```
1. CREATE TABLE cats3
2. (
3.     name VARCHAR(20) DEFAULT 'no name provided',
4.     age INT DEFAULT 99
5. );
```

Notice the change when you describe the table: `DESC cats3;`

Insert a cat without a name: `INSERT INTO cats3 (age) VALUES (13);`

Or a nameless, ageless cat: `INSERT INTO cats3 () VALUES ();`

Combine NOT NULL and DEFAULT:

```
1. CREATE TABLE cats4
2. (
3.     name VARCHAR(20) NOT NULL DEFAULT 'unnamed',
4.     age INT NOT NULL DEFAULT 99
5. );
```

Notice The Difference:

```
1. INSERT INTO cats () VALUES ();
2.
3. SELECT * FROM cats;
4.
5. INSERT INTO cats3 () VALUES ();
6.
7. SELECT * FROM cats3;
8.
9. INSERT INTO cats3 (name, age) VALUES ('Montana', NULL);
10.
11. SELECT * FROM cats3;
12.
13. INSERT INTO cats4 (name, age) VALUES ('Cali', NULL);
```

PRIMARY KEYS

Define a table with a **PRIMARY KEY** constraint:

```
1. CREATE TABLE unique_cats
2. (
3.     cat_id INT NOT NULL,
4.     name VARCHAR(100),
5.     age INT,
6.     PRIMARY KEY (cat_id)
7. );
```

`DESC unique_cats;`

Insert some new cats:

```
1. INSERT INTO unique_cats(cat_id, name, age) VALUES(1, 'Fred', 23);
2.
3. INSERT INTO unique_cats(cat_id, name, age) VALUES(2, 'Louise', 3);
4.
5. INSERT INTO unique_cats(cat_id, name, age) VALUES(1, 'James', 3);
```

Notice what happens:

```
SELECT * FROM unique_cats;
```

Adding in **AUTO_INCREMENT**:

```
1. CREATE TABLE unique_cats2 (
2.     cat_id INT NOT NULL AUTO_INCREMENT,
3.     name VARCHAR(100),
4.     age INT,
5.     PRIMARY KEY (cat_id)
6. );
```

INSERT a couple new cats:

```
1. INSERT INTO unique_cats2(name, age) VALUES('Skippy', 4);
2. INSERT INTO unique_cats2(name, age) VALUES('Jiff', 3);
3. INSERT INTO unique_cats2(name, age) VALUES('Jiff', 3);
4. INSERT INTO unique_cats2(name, age) VALUES('Jiff', 3);
5. INSERT INTO unique_cats2(name, age) VALUES('Skippy', 4);
```

Notice the difference:

```
SELECT * FROM unique_cats2;
```

TABLE CONSTRAINTS EXERCISE

Define an Employees table, with the following fields:

- **id** - number(automatically increments), mandatory, primary key
- **last_name** - text, mandatory
- **first_name** - text, mandatory
- **middle_name** - text, not mandatory
- **age** - number mandatory
- **current_status** - text, mandatory, defaults to 'employed'

Defining The employees table:

```
1. CREATE TABLE employees (
2.     id INT AUTO_INCREMENT NOT NULL,
3.     first_name VARCHAR(255) NOT NULL,
4.     last_name VARCHAR(255) NOT NULL,
5.     middle_name VARCHAR(255),
6.     age INT NOT NULL,
7.     current_status VARCHAR(255) NOT NULL DEFAULT 'employed',
8.     PRIMARY KEY (id)
9. );
```

Another way of defining a primary key:

```
1. CREATE TABLE employees (
```

```

2.      id INT AUTO_INCREMENT NOT NULL PRIMARY KEY,
3.      first_name VARCHAR(255) NOT NULL,
4.      last_name VARCHAR(255) NOT NULL,
5.      middle_name VARCHAR(255),
6.      age INT NOT NULL,
7.      current_status VARCHAR(255) NOT NULL DEFAULT 'employed'
8. );

```

A test INSERT:

```

1. INSERT INTO employees(first_name, last_name, age) VALUES
2. ('Dora', 'Smith', 58);

```

CRUD=CREATE READ UPDATE DELETE

PREPARING DATA

Let's drop the existing cats table:

```
DROP TABLE cats;
```

Recreate a new cats table:

```

1. CREATE TABLE cats
2. (
3.     cat_id INT NOT NULL AUTO_INCREMENT,
4.     name    VARCHAR(100),
5.     breed   VARCHAR(100),
6.     age     INT,
7.     PRIMARY KEY (cat_id)
8. );

```

```
DESC cats;
```

And finally insert some new cats:

```

1. INSERT INTO cats(name, breed, age)
2. VALUES ('Ringo', 'Tabby', 4),
3.         ('Cindy', 'Maine Coon', 10),
4.         ('Dumbledore', 'Maine Coon', 11),
5.         ('Egg', 'Persian', 4),
6.         ('Misty', 'Tabby', 13),
7.         ('George Michael', 'Ragdoll', 9),
8.         ('Jackson', 'Sphynx', 7);

```

SELECT

```
SELECT * FROM cats;
```

```
SELECT cat_id FROM cats;
```

```
SELECT cat_id, name, age, breed FROM cats;
```

WHERE

Select by age:

```
SELECT * FROM cats WHERE age=4;
```

Select by name:

```
SELECT * FROM cats WHERE name='Egg';
```

Notice how it deals with case: matter)

```
SELECT * FROM cats WHERE name='egG'; (does not
```

```
SELECT cat_id, age FROM cats WHERE cat_id=age;
```

ALIASES

```
1. SELECT name AS 'cat name', breed AS 'kitty breed' FROM cats;
```

cat name	kitty breed
Ringo	Tabby
Cindy	Maine Coon
Dumbledore	Maine Coon
Egg	Persian
Misty	Tabby
George Michael	Ragdoll
Jackson	Sphynx

```
2. DESC cats;
```

Field	Type	Null	Key	Default	Extra
cat_id	int(11)	NO	PRI	NULL	auto_increment
name	varchar(100)	YES		NULL	
breed	varchar(100)	YES		NULL	
age	int(11)	YES		NULL	

UPDATING DATA

Change tabby cats to shorthair:
breed='Tabby';

```
UPDATE cats SET breed='Shorthair' WHERE
```

Another update:

```
UPDATE cats SET age=14 WHERE name='Misty';
```

Rule of thumb:
data, no undo button)

SELECT and then UPDATE (make sure we target the right

UPDATE CHALLENGES

1. **SELECT** * **FROM** cats **WHERE** name='Jackson';
- 2.
3. **UPDATE** cats **SET** name='Jack' **WHERE** name='Jackson';
- 4.
5. **SELECT** * **FROM** cats **WHERE** name='Ringo';
- 6.
7. **UPDATE** cats **SET** breed='British Shorthair' **WHERE** name='Ringo';

DELETING DATA

1. **DELETE FROM** cats **WHERE** name='Egg';
- 2.
3. **DELETE FROM** cats; (All data in cats would be gone.)

DELETE CHALLENGES SOLUTION

1. **SELECT** * **FROM** cats **WHERE** age=4;
2. **DELETE FROM** cats **WHERE** age=4;
3. **SELECT** * **FROM** cats **WHERE** age=4;
4. **SELECT** * **FROM** cats;
5. **SELECT** * **FROM** cats **WHERE** cat_id=age;

```
6. DELETE FROM cats WHERE cat_id=age;
7. DELETE FROM cats;
8. SELECT * FROM cats;
```

All deleted!

SPRING CLEANING – THE ANNUAL CLOSET INVENTORY –CRUD EXERCISE

```
SELECT DATABASE ();
```

```
CREATE DATABASE shirts_db;
```

```
SHOW DATABASE
```

```
USE shirts_db;
```

```
CREATE TABLE shirts (  
shirt_id INT AUTO_INCREMENT NOT NULL PRIMARY KEY,  
article VARCHAR(100),  
color VARCHAR(100),  
shirt_size VARCHAR(2),  
last_worn INT);
```

```
DESC shirts;
```

```
INSERT INTO shirts (article, color, shirt_size,last_worn)
```

```
VALUES
```

```
('t-shirt', 'white', 'S', 10), ('t-shirt', 'green', 'S', 200),  
('polo shirt', 'black', 'M', 10), ('tank top', 'blue', 'S', 50),  
('t-shirt', 'pink', 'S', 0), ('polo shirt', 'red', 'M', 5),  
('tank top', 'white', 'S', 200), ('tank top', 'blue', 'M', 15),  
('polo shirt', 'purple', 'M', 50);
```

```
SELECT * FROM shirts WHERE article='polo shirt';
```

```
UPDATE shirts SET shirt_size='L' WHERE article='polo shirt';
```

```
SELECT * FROM shirts WHERE last_worn=15;
```

```
UPDATE shirts SET last_worn=0 WHERE last_worn=15;
```

```
SELECT * FROM shirts WHERE color='white';
```

```

UPDATE shirts SET shirt_size='XS' WHERE color='white';

UPDATE shirts SET color='off white' WHERE color='white';


DELETE FROM shirts WHERE last_worn=200;

DELETE FROM shirts WHERE article='tank top';

DELETE FROM shirts;


DROP TABLE shirts;

SHOW TABLE shirts;

DESC shirts;


DROP DATABASE shirts_db;

SELECT * FROM shirts;

```

THE WORLD OF STRING FUNCTIONS

RUNNING SQL FILES

```

1. CREATE TABLE cats
2.     (   cat_id INT NOT NULL AUTO_INCREMENT,
3.         name VARCHAR(100),
4.         age INT,
5.         PRIMARY KEY(cat_id) );
6. mysql-ctl cli
7. use cat_app;
8. source first_file.sql
9.
10. DESC cats;
11.
12. INSERT INTO cats(name, age)
13. VALUES('Charlie', 17);
14. INSERT INTO cats(name, age)
15. VALUES('Connie', 10);
16.
17. SELECT * FROM cats;
18.
19. source testing/insert.sql

```

LOADING BOOK DATA

1. FIRST CREATE YOUR BOOK_DATA.SQL FILE WITH THE FOLLOWING CODE:

```

DROP DATABASE IF EXISTS book_shop;

CREATE DATABASE book_shop;

USE book_shop;


CREATE TABLE books (

```

```

        book_id INT NOT NULL AUTO_INCREMENT,

        title VARCHAR(100),

        author_fname VARCHAR(100),

        author_lname VARCHAR(100),

        released_year INT,

        stock_quantity INT,

        pages INT,

        PRIMARY KEY(book_id)

    );

INSERT INTO books (title, author_fname, author_lname, released_year,
stock_quantity, pages)

VALUES

('The Namesake', 'Jhumpa', 'Lahiri', 2003, 32, 291),

('Norse Mythology', 'Neil', 'Gaiman', 2016, 43, 304),

('American Gods', 'Neil', 'Gaiman', 2001, 12, 465),

('Interpreter of Maladies', 'Jhumpa', 'Lahiri', 1996, 97, 198),

('A Hologram for the King: A Novel', 'Dave', 'Eggers', 2012, 154, 352),

('The Circle', 'Dave', 'Eggers', 2013, 26, 504),

('The Amazing Adventures of Kavalier & Clay', 'Michael', 'Chabon', 2000, 68,
634),

('Just Kids', 'Patti', 'Smith', 2010, 55, 304),

('A Heartbreaking Work of Staggering Genius', 'Dave', 'Eggers', 2001, 104,
437),

('Coraline', 'Neil', 'Gaiman', 2003, 100, 208),

('What We Talk About When We Talk About Love: Stories', 'Raymond', 'Carver',
1981, 23, 176),

('Where I'm Calling From: Selected Stories', 'Raymond', 'Carver', 1989, 12,
526),

('White Noise', 'Don', 'DeLillo', 1985, 49, 320),

('Cannery Row', 'John', 'Steinbeck', 1945, 95, 181),

('Oblivion: Stories', 'David', 'Foster Wallace', 2004, 172, 329),

('Consider the Lobster', 'David', 'Foster Wallace', 2005, 92, 343);

```

2. THEN SOURCE THAT FILE

source book_data.sql

3. NOW CHECK YOUR WORK:

1. **DESC** books;
2. **SELECT * FROM** books;

CONCAT

1. **SELECT** author_fname, author_lname **FROM** books;
- 2.
3. **CONCAT(author_fname, author_lname); // invalid syntax**
- 4.
5. **SELECT CONCAT**('Hello', 'World');
- 6.
7. **SELECT**
8. **CONCAT**(author_fname, ' ', author_lname)
9. **FROM** books;
- 10.
11. **SELECT**
12. **CONCAT**(author_fname, ' ', author_lname)
13. **AS** 'full name'
14. **FROM** books;
- 15.
16. **SELECT** author_fname **AS** first, author_lname **AS** last,
17. **CONCAT**(author_fname, ' ', author_lname) **AS** full
18. **FROM** books;
- 19.
20. **SELECT** author_fname **AS** first, author_lname **AS** last,
21. **CONCAT**(author_fname, ' ', author_lname) **AS** full
22. **FROM** books;
- 23.
24. **SELECT CONCAT**(title, '-', author_fname, '-', author_lname) **FROM** books;
- 25.
26. **SELECT**
27. **CONCAT_WS**(' - ', title, author_fname, author_lname)
28. **FROM** books;

SUBSTRING

1. **SELECT SUBSTRING**('Hello World', 1, 4);
- 2.
3. **SELECT SUBSTRING**('Hello World', 7);
- 4.
5. **SELECT SUBSTRING**('Hello World', 3, 8);
- 6.
7. **SELECT SUBSTRING**('Hello World', 3);
- 8.
9. **SELECT SUBSTRING**('Hello World', -3);
- 10.
11. **SELECT SUBSTRING**('Hello World', -7);
- 12.
13. **SELECT** title **FROM** books;
- 14.
15. **SELECT SUBSTRING**("Where I'm Calling From: Selected Stories", 1, 10);
- 16.
17. **SELECT SUBSTRING**(title, 1, 10) **FROM** books;
- 18.
19. **SELECT SUBSTRING**(title, 1, 10) **AS** 'short title' **FROM** books;
- 20.
21. **SELECT SUBSTR**(title, 1, 10) **AS** 'short title' **FROM** books;

```

22.
23. SELECT CONCAT
24.     (
25.         SUBSTRING(title, 1, 10),
26.         '...'
27.     )
28. FROM books;
29.
30. source book_code.sql
31.
32. SELECT CONCAT
33.     (
34.         SUBSTRING(title, 1, 10),
35.         '...'
36.     ) AS 'short title'
37. FROM books;
38.
39. source book_code.sql

```

REPLACE

```

1. SELECT REPLACE('Hello World', 'Hell', '%$#@');
2.
3. SELECT REPLACE('Hello World', 'l', '7');
4.
5. SELECT REPLACE('Hello World', 'o', '0');
6.
7. SELECT REPLACE('Hello World', 'o', '*');
8.
9. SELECT
10.     REPLACE('cheese bread coffee milk', ' ', ' and ');
11.
12. SELECT REPLACE(title, 'e ', '3') FROM books;
13.
14. -- SELECT
15. --     CONCAT
16. --     (
17. --         SUBSTRING(title, 1, 10),
18. --         '...'
19. --     ) AS 'short title'
20. -- FROM books;
21.
22. SELECT
23.     SUBSTRING(
24.         REPLACE(title, 'e', '3'),
25.         1,
26.         10)
27. FROM books;
28.
29. SELECT
30.     SUBSTRING(REPLACE(title, 'e', '3'), 1, 10) AS 'weird string'
31. FROM books;

```

Notes:

- Use `cmd + /` (mac) or `ctrl + /` (pc) to comment out SQL in c9.
- The `REPLACE()` function, as well as the other string functions, only change the query output, they don't affect the actual data in the database.

REVERSE

```

1. SELECT REVERSE('meow meow');
2.
3. SELECT REVERSE(author_fname) FROM books;
4.
5. SELECT CONCAT('woof', REVERSE('woof'));

```

- 6.
7. **SELECT CONCAT**(author_fname, **REVERSE**(author_fname)) **FROM** books;

CHAR LENGTH

1. **SELECT** CHAR_LENGTH('Hello World');
2. **SELECT** author_lname, **CHAR_LENGTH**(author_lname) **AS** 'length' **FROM** books;
- 3.
4. **SELECT CONCAT**(author_lname, ' is ', CHAR_LENGTH(author_lname), ' characters long') **FROM** books;

CHANGING CASE WITH UPPER AND LOWER

1. **SELECT** **UPPER**('Hello World');
- 2.
3. **SELECT** **LOWER**('Hello World');
- 4.
5. **SELECT** **UPPER**(title) **FROM** books;
- 6.
7. **SELECT** **CONCAT**('MY FAVORITE BOOK IS ', **UPPER**(title)) **FROM** books;
- 8.
9. **SELECT** **CONCAT**('MY FAVORITE BOOK IS ', **LOWER**(title)) **FROM** books;

NOTE ABOUT STRING FUNCTIONS

This works:

1. **SELECT** **UPPER**(**CONCAT**(author_fname, ' ', author_lname)) **AS** "full name in caps"
2. **FROM** books;

While this does not:

1. **SELECT** **CONCAT**(**UPPER**(author_fname, ' ', author_lname)) **AS** "full name in caps"
2. **FROM** books;

You could do it this way, however:

1. **SELECT** **CONCAT**(**UPPER**(author_fname), ' ', **UPPER**(author_lname)) **AS** "full name in caps"
2. **FROM** books;

STRING FUNCTION CHALLENGES

-- Concatenate with a separator 'CONCAT_WS'

```
SELECT author_fname as firstname, author_lname as lastname,  
  
CONCAT_WS (' ', author_fname, author_lname) as fullname FROM books;
```

```
SELECT REVERSE(UPPER('Why does my cat look at me with such hatred?'));
```

```
SELECT UPPER(REVERSE('Why does my cat look at me with such hatred?'));
```

```
SELECT REPLACE(CONCAT('I', ' ', 'like', ' ', 'cats'), ' ', '-');
```

```
SELECT CONCAT(author_lname, ' is ', CHAR_LENGTH(author_lname), ' characters long') FROM books;
```

```
SELECT CONCAT('MY FAVORITE BOOK IS ', UPPER(title)) FROM books;
```

```
SELECT CONCAT('MY FAVORITE BOOK IS ', LOWER(title)) FROM books;
```

```
-- forwards & backwards
```

```
SELECT REPLACE(title, ' ', '_') as title FROM books;
```

```
SELECT author_lname as forwards, REVERSE(author_lname) as backwards FROM books;
```

```
-- --Full Name in Caps
```

```
SELECT UPPER(CONCAT(author_fname, ' ', author_lname)) as 'full name in caps' FROM books;
```

```
SELECT CONCAT(title, ' was released in ', released_year) as blurb FROM books;
```

```
-- Print book titles and the length of each title
```

```
SELECT title, CHAR_LENGTH(title) as 'character count' FROM books;
```

```
SELECT
```

```
CONCAT(SUBSTR(title,1,10),'...') as 'short title',
```

```
CONCAT(author_lname,', ',author_fname) as author,
```

```
CONCAT(stock_quantity, ' in stock ') as quantity FROM books;
```

DISTINCT

1. **SELECT** author_lname **FROM** books;
- 2.
3. **SELECT** **DISTINCT** author_lname **FROM** books;
- 4.
5. **SELECT** author_fname, author_lname **FROM** books;
- 6.
7. **SELECT** **DISTINCT** **CONCAT**(author_fname, ' ', author_lname) **FROM** books;
- 8.
9. **SELECT** **DISTINCT** author_fname, author_lname **FROM** books;

SORTING DATA WITH ORDER BY

1. **SELECT** author_lname **FROM** books;
- 2.
3. **SELECT** author_lname **FROM** books **ORDER BY** author_lname;
- 4.
5. **SELECT** title **FROM** books;
- 6.
7. **SELECT** title **FROM** books **ORDER BY** title;
8. **SELECT** author_lname **FROM** books **ORDER BY** author_lname **DESC**;

```

9.
10. SELECT released_year FROM books;
11.
12. SELECT released_year FROM books ORDER BY released_year;
13.
14. SELECT released_year FROM books ORDER BY released_year DESC;
15.
16. SELECT released_year FROM books ORDER BY released_year ASC;
17.
18. SELECT title, released_year, pages FROM books ORDER BY released_year;
19.
20. SELECT title, pages FROM books ORDER BY released_year;
21.
22. SELECT title, author_fname, author_lname
23. FROM books ORDER BY 2;
24.
25. SELECT title, author_fname, author_lname
26. FROM books ORDER BY 3;
27.
28. SELECT title, author_fname, author_lname
29. FROM books ORDER BY 1;
30.
31. SELECT title, author_fname, author_lname
32. FROM books ORDER BY 1 DESC;
33.
34. SELECT author_lname, title
35. FROM books ORDER BY 2;
36.
37. SELECT author_fname, author_lname FROM books
38. ORDER BY author_lname, author_fname;

```

LIMIT

```

1. SELECT title FROM books LIMIT 3;
2.
3. SELECT title FROM books LIMIT 1;
4.
5. SELECT title FROM books LIMIT 10;
6.
7. SELECT * FROM books LIMIT 1;
8.
9. SELECT title, released_year FROM books
10. ORDER BY released_year DESC LIMIT 5;
11.
12. SELECT title, released_year FROM books
13. ORDER BY released_year DESC LIMIT 1;
14.
15. SELECT title, released_year FROM books
16. ORDER BY released_year DESC LIMIT 14;
17.
18. SELECT title, released_year FROM books
19. ORDER BY released_year DESC LIMIT 0,5;
20.
21. SELECT title, released_year FROM books
22. ORDER BY released_year DESC LIMIT 0,3;
23.
24. SELECT title, released_year FROM books
25. ORDER BY released_year DESC LIMIT 1,3;
26.
27. SELECT title, released_year FROM books
28. ORDER BY released_year DESC LIMIT 10,1;
29.
30. SELECT * FROM tbl LIMIT 95,18446744073709551615;
31.
32. SELECT title FROM books LIMIT 5;
33.
34. SELECT title FROM books LIMIT 5, 123219476457;
35.

```

36. `SELECT title FROM books LIMIT 5, 50;`

SEARCHES WITH LIKE - WILDCARD

1. `SELECT title, author_fname FROM books WHERE author_fname LIKE '%da%';`
2. `SELECT title, author_fname FROM books WHERE author_fname LIKE 'da%';`
3. `SELECT title FROM books WHERE title LIKE 'the';`
4. `SELECT title FROM books WHERE title LIKE '%the';`
5. `SELECT title FROM books WHERE title LIKE '%the%';`

LIKE PART 2: MORE WILDCARDS

1. `SELECT title, stock_quantity FROM books;`
- 2.
3. `SELECT title, stock_quantity FROM books WHERE stock_quantity LIKE '____';`
- 4.
5. `SELECT title, stock_quantity FROM books WHERE stock_quantity LIKE '___';`
- 6.
7. `(235)234-0987 LIKE '(____)____-____';`
- 8.
9. `SELECT title FROM books;`
- 10.
11. `SELECT title FROM books WHERE title LIKE '%\%%'`
- 12.
13. `SELECT title FROM books WHERE title LIKE '%\_%'`

`\` is a special character known as an escape character that indicates that the character directly following it should be interpreted literally (useful for single quotes, wildcard characters, etc.).

REFINING SELECTIONS EXERCISES

`SELECT title FROM books WHERE title LIKE '%stories%';`

`SELECT title, pages FROM books ORDER BY char_length(title) DESC LIMIT 1;`

`SELECT title, pages FROM books ORDER BY pages DESC LIMIT 1;`

`SELECT CONCAT(title, ' - ', released_year) AS summary FROM books ORDER BY released_year DESC LIMIT 3;`

`SELECT title, released_year, stock_quantity FROM books ORDER BY stock_quantity, released_year ASC LIMIT 3;`

`SELECT title, author_lname FROM books WHERE author_lname LIKE '% %';`

`SELECT title, author_lname FROM books ORDER BY 2, 1;`

`SELECT UPPER(CONCAT('my favorite author is ', author_fname, ', ', author_lname, '!')) AS yell FROM books ORDER BY author_lname;`

THE MAGIC OF AGGREGATE FUNCTIONS

THE COUNT FUNCTION

1. `SELECT COUNT(*) FROM books;`
- 2.
3. `SELECT COUNT(author_fname) FROM books;`

```

4.
5. SELECT COUNT(DISTINCT author_fname) FROM books;
6.
7. SELECT COUNT(DISTINCT author_lname) FROM books;
8.
9. SELECT COUNT(DISTINCT author_lname, author_fname) FROM books;
10.
11. SELECT title FROM books WHERE title LIKE '%the%';
12.
13. SELECT COUNT(*) FROM books WHERE title LIKE '%the%';

```

THE JOYS OF GROUP BY

```

1. SELECT title, author_lname FROM books;
2.
3. SELECT title, author_lname FROM books
4. GROUP BY author_lname;
5.
6. SELECT author_lname, COUNT(*)
7. FROM books GROUP BY author_lname;
8.
9.
10. SELECT title, author_fname, author_lname FROM books;
11.
12. SELECT title, author_fname, author_lname FROM books GROUP BY
    author_lname;
13.
14. SELECT author_fname, author_lname, COUNT(*) FROM books GROUP BY
    author_lname;
15.
16. SELECT author_fname, author_lname, COUNT(*) FROM books GROUP BY
    author_lname, author_fname;
17.
18. SELECT released_year FROM books;
19.
20. SELECT released_year, COUNT(*) FROM books GROUP BY released_year;
21.
22. SELECT CONCAT('In ', released_year, ' ', COUNT(*), ' book(s) released')
    AS year FROM books GROUP BY released_year;

```

MIN AND MAX BASICS

```

1. SELECT MIN(released_year)
2. FROM books;
3.
4. SELECT MIN(released_year) FROM books;
5.
6. SELECT MIN(pages) FROM books;
7.
8. SELECT MAX(pages)
9. FROM books;
10.
11. SELECT MAX(released_year)
12. FROM books;

```

A PROBLEM WITH MIN AND MAX

```

1. SELECT * FROM books
2. WHERE pages = (SELECT Min(pages)
3.                FROM books);
4.
5. SELECT title, pages FROM books
6. WHERE pages = (SELECT Max(pages)
7.                FROM books);
8.

```

```

9. SELECT title, pages FROM books
10. WHERE pages = (SELECT Min(pages)
11.                FROM books);
12.
13. SELECT * FROM books
14. ORDER BY pages ASC LIMIT 1;
15.
16. SELECT title, pages FROM books
17. ORDER BY pages ASC LIMIT 1;
18.
19. SELECT * FROM books
20. ORDER BY pages DESC LIMIT 1;

```

MIN AND MAX WITH GROUP BY

```

1. SELECT author_fname,
2.        author_lname,
3.        Min(released_year)
4. FROM   books
5. GROUP BY author_lname,
6.          author_fname;
7.
8. SELECT
9.    author_fname,
10.   author_lname,
11.   Max(pages)
12. FROM books
13. GROUP BY author_lname,
14.          author_fname;
15.
16. SELECT
17.    CONCAT(author_fname, ' ', author_lname) AS author,
18.    MAX(pages) AS 'longest book'
19. FROM books
20. GROUP BY author_lname,
21.          author_fname;

```

THE SUM FUNCTION

```

1. SELECT SUM(pages)
2. FROM books;
3.
4. SELECT SUM(released_year) FROM books;
5.
6. SELECT author_fname,
7.        author_lname,
8.        Sum(pages)
9. FROM books
10. GROUP BY
11.    author_lname,
12.    author_fname;
13.
14. SELECT author_fname,
15.        author_lname,
16.        Sum(released_year)
17. FROM books
18. GROUP BY
19.    author_lname,
20.    author_fname;

```

THE AVG FUNCTION

```

1. SELECT AVG(released_year)
2. FROM books;
3.

```

```

4. SELECT AVG(pages)
5. FROM books;
6.
7. SELECT AVG(stock_quantity)
8. FROM books
9. GROUP BY released_year;
10.
11. SELECT released_year, AVG(stock_quantity)
12. FROM books
13. GROUP BY released_year;
14.
15. SELECT author_fname, author_lname, AVG(pages) FROM books
16. GROUP BY author_lname, author_fname;

```

AGGREGATE FUNCTIONS CHALLENGES

```

1. SELECT COUNT(*) FROM books;
2.
3. SELECT COUNT(*) FROM books GROUP BY released_year;
4.
5. SELECT released_year, COUNT(*) FROM books GROUP BY released_year;
6.
7. SELECT Sum(stock_quantity) FROM BOOKS;
8.
9. SELECT AVG(released_year) FROM books GROUP BY author_lname, author_fname;
10.
11. SELECT author_fname, author_lname, AVG(released_year) FROM books GROUP
    BY author_lname, author_fname;
12.
13. SELECT CONCAT(author_fname, ' ', author_lname) FROM books
14. WHERE pages = (SELECT Max(pages) FROM books);
15.
16. SELECT CONCAT(author_fname, ' ', author_lname) FROM books
17. ORDER BY pages DESC LIMIT 1;
18.
19. SELECT pages, CONCAT(author_fname, ' ', author_lname) FROM books
20. ORDER BY pages DESC;
21.
22. SELECT released_year AS year,
23.       COUNT(*) AS '# of books',
24.       AVG(pages) AS 'avg pages'
25. FROM books
26.     GROUP BY released_year;

```

DATA TYPES

CHAR AND VARCHAR

```

1. CREATE TABLE dogs (name CHAR(5), breed VARCHAR(10));
2.
3. INSERT INTO dogs (name, breed) VALUES ('bob', 'beagle');
4.
5. INSERT INTO dogs (name, breed) VALUES ('robby', 'corgi');
6.
7. INSERT INTO dogs (name, breed) VALUES ('Princess Jane', 'Retriever');
8.
9. SELECT * FROM dogs;
10.
11. INSERT INTO dogs (name, breed) VALUES ('Princess Jane',
    'Retrievesadfdsafdasfsafr');
12.
13. SELECT * FROM dogs;

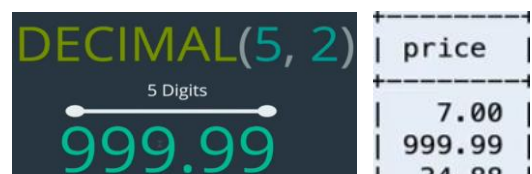
```

DECIMAL

```

1. CREATE TABLE items(price DECIMAL(5,2));

```



```

2.
3. INSERT INTO items(price) VALUES (7);
4.
5. INSERT INTO items(price) VALUES (7987654);
6.
7. INSERT INTO items(price) VALUES (34.88);
8.
9. INSERT INTO items(price) VALUES (298.9999);
10.
11. INSERT INTO items(price) VALUES (1.9999);
12.
13. SELECT * FROM items;

```

Data Type	Memory Needed	Precision Issues
FLOAT	4 Bytes	~7 digits
DOUBLE	8 Bytes	~15 digits

The DECIMAL data type is a fixed-point type and calculations are exact. In MySQL, the DECIMAL type has several synonyms: NUMERIC, DEC, FIXED. The integer types also are exact-value types.

The FLOAT and DOUBLE data types are floating-point types and calculations are approximate. In MySQL, types that are synonymous with FLOAT or DOUBLE are DOUBLE PRECISION and REAL.

FLOAT AND DOUBLE

```

1. CREATE TABLE thingies (price FLOAT);
2.
3. INSERT INTO thingies(price) VALUES (88.45);
4.
5. SELECT * FROM thingies;
6.
7. INSERT INTO thingies(price) VALUES (8877.45);
8.
9. SELECT * FROM thingies;
10.
11. INSERT INTO thingies(price) VALUES (8877665544.45);
12.
13. SELECT * FROM thingies;

```

price
88.45
8877.45
8877670000

PRECISION: DECIMAL > DOUBLE > FLOAT

DATE DATA

```

1. CREATE TABLE people (name VARCHAR(100), birthdate DATE, birthtime TIME,
   birthdt DATETIME);
2.
3. INSERT INTO people (name, birthdate, birthtime, birthdt)
4. VALUES('Padma', '1983-11-11', '10:07:35', '1983-11-11 10:07:35');
5.
6. INSERT INTO people (name, birthdate, birthtime, birthdt)
7. VALUES('Larry', '1943-12-25', '04:10:42', '1943-12-25 04:10:42');
8.
9. SELECT * FROM people;

```

FORMATTING DATES

```

1. SELECT name, birthdate FROM people;
2.
3. SELECT name, DAY(birthdate) FROM people;
4.
5. SELECT name, birthdate, DAY(birthdate) FROM people;
6.
7. SELECT name, birthdate, DAYNAME(birthdate) FROM people;

```

```

8.
9. SELECT name, birthdate, DAYOFWEEK(birthdate) FROM people;
10.
11. SELECT name, birthdate, DAYOFYEAR(birthdate) FROM people;
12.
13. SELECT name, birthtime, DAYOFYEAR(birthtime) FROM people;
14.
15. SELECT name, birthdt, DAYOFYEAR(birthdt) FROM people;
16.
17. SELECT name, birthdt, MONTH(birthdt) FROM people;
18.
19. SELECT name, birthdt, MONTHNAME(birthdt) FROM people;
20.
21. SELECT name, birthtime, HOURL(birthtime) FROM people;
22.
23. SELECT name, birthtime, MINUTE(birthtime) FROM people;
24.
25. SELECT CONCAT(MONTHNAME(birthdate), ' ', DAY(birthdate), ' ',
    YEAR(birthdate)) FROM people;
26.
27. SELECT DATE_FORMAT(birthdt, 'Was born on a %W') FROM people;
28. SELECT DATE_FORMAT(birthdt, '%m/%d/%Y') FROM people;
29. SELECT DATE_FORMAT(birthdt, '%m/%d/%Y at %h:%i') FROM people;

```

DATE MATH

```

1. SELECT * FROM people;
2.
3. SELECT DATEDIFF(NOW(), birthdate) FROM people;
4.
5. SELECT name, birthdate, DATEDIFF(NOW(), birthdate) FROM people;
6.
7. SELECT birthdt FROM people;
8.
9. SELECT birthdt, DATE_ADD(birthdt, INTERVAL 1 MONTH) FROM people;
10.
11. SELECT birthdt, DATE_ADD(birthdt, INTERVAL 10 SECOND) FROM people;
12.
13. SELECT birthdt, DATE_ADD(birthdt, INTERVAL 3 QUARTER) FROM people;
14.
15. SELECT birthdt, birthdt + INTERVAL 1 MONTH FROM people;
16.
17. SELECT birthdt, birthdt - INTERVAL 5 MONTH FROM people;
18.
19. SELECT birthdt, birthdt + INTERVAL 15 MONTH + INTERVAL 10 HOUR FROM
    people;

```

WORKING WITH TIMESTAMPS

```

1. CREATE TABLE comments (
2.     content VARCHAR(100),
3.     created_at TIMESTAMP DEFAULT NOW()
4. );
5.
6. INSERT INTO comments (content) VALUES('lol what a funny article');
7.
8. INSERT INTO comments (content) VALUES('I found this offensive');
9.
10. INSERT INTO comments (content) VALUES('Ifasfsadfsadfsad');
11.
12. SELECT * FROM comments ORDER BY created_at DESC;
13.
14. CREATE TABLE comments2 (
15.     content VARCHAR(100),
16.     changed_at TIMESTAMP DEFAULT NOW() ON UPDATE CURRENT_TIMESTAMP
17. );
18.

```

```

19. INSERT INTO comments2 (content) VALUES('dasdasdasd');
20.
21. INSERT INTO comments2 (content) VALUES('lolololololo');
22.
23. INSERT INTO comments2 (content) VALUES('I LIKE CATS AND DOGS');
24.
25. UPDATE comments2 SET content='THIS IS NOT GIBBERISH' WHERE
    content='dasdasdasd';
26.
27. SELECT * FROM comments2;
28.
29. SELECT * FROM comments2 ORDER BY changed_at;
30.
31. CREATE TABLE comments2 (
32.     content VARCHAR(100),
33.     changed_at TIMESTAMP DEFAULT NOW() ON UPDATE NOW()
34. );

```

Main Difference: Range of the date and time

The **DATETIME** type is used for values that contain both date and time parts. MySQL retrieves and displays **DATETIME** values in 'YYYY-MM-DD HH:MM:SS' format. The supported range is '1000-01-01 00:00:00' to '9999-12-31 23:59:59'.

The **TIMESTAMP** data type is used for values that contain both date and time parts. **TIMESTAMP** has a range of '1970-01-01 00:00:01' UTC to '2038-01-19 03:14:07' UTC.

DATA TYPES EXERCISES

```

1. What's a good use case for CHAR?
2. -----
3. Used for text that we know has a fixed length, e.g., State abbreviations,
4. abbreviated company names, sex M/F, etc.
5.
6. CREATE TABLE inventory (
7.     item_name VARCHAR(100),
8.     price DECIMAL(8,2),
9.     quantity INT
10. );
11.
12. What's the difference between DATETIME and TIMESTAMP?
13. -----
14. They both store datetime information, but there's a difference in the
    range,
15. TIMESTAMP has a smaller range. TIMESTAMP also takes up less space.
16. TIMESTAMP is used for things like meta-data about when something is
    created
17. or updated.
18.
19. SELECT CURTIME();
20.
21. SELECT CURDATE();
22.
23. SELECT DAYOFWEEK(CURDATE());
24. SELECT DAYOFWEEK(NOW());
25. SELECT DATE_FORMAT(NOW(), '%w') + 1;
26.
27. SELECT DAYNAME(NOW());
28. SELECT DATE_FORMAT(NOW(), '%W');
29.
30. SELECT DATE_FORMAT(CURDATE(), '%m/%d/%Y');
31.
32. SELECT DATE_FORMAT(NOW(), '%M %D at %h:%i');
33.
34. CREATE TABLE tweets(

```

```

35.     content VARCHAR(140),
36.     username VARCHAR(20),
37.     created_at TIMESTAMP DEFAULT NOW()
38. );
39.
40. INSERT INTO tweets (content, username) VALUES('this is my first tweet',
    'coltscat');
41. SELECT * FROM tweets;
42.
43. INSERT INTO tweets (content, username) VALUES('this is my second
    tweet', 'coltscat');
44. SELECT * FROM tweets;

```

DATE AND TIME FUNCTION DOCUMENTATION

<https://dev.mysql.com/doc/refman/8.0/en/date-and-time-functions.html>

THE POWER OF LOGICAL OPERATORS

NOT EQUAL

```

1. SELECT title FROM books WHERE released_year = 2017;
2.
3. SELECT title FROM books WHERE released_year != 2017;
4.
5. SELECT title, author_lname FROM books;
6.
7. SELECT title, author_lname FROM books WHERE author_lname = 'Harris';
8.
9. SELECT title, author_lname FROM books WHERE author_lname != 'Harris';

```

NOT LIKE

```

1. SELECT title FROM books WHERE title LIKE 'W';
2.
3. SELECT title FROM books WHERE title LIKE 'W%';
4.
5. SELECT title FROM books WHERE title LIKE '%W%';
6.
7. SELECT title FROM books WHERE title LIKE 'W%';
8.
9. SELECT title FROM books WHERE title NOT LIKE 'W%';

```

GREATER THAN

```

1. SELECT title, released_year FROM books ORDER BY released_year;
2.
3. SELECT title, released_year FROM books
4. WHERE released_year > 2000 ORDER BY released_year;
5.
6. SELECT title, released_year FROM books
7. WHERE released_year >= 2000 ORDER BY released_year;
8.
9. SELECT title, stock_quantity FROM books;
10.
11. SELECT title, stock_quantity FROM books WHERE stock_quantity >= 100;
12.
13. SELECT 99 > 1;
14.
15. SELECT 99 > 567;
16.
17. 100 > 5

```

```

18. -- true
19.
20. -15 > 15
21. -- false
22.
23. 9 > -10
24. -- true
25.
26. 1 > 1
27. -- false
28.
29. 'a' > 'b'
30. -- false
31.
32. 'A' > 'a'
33. -- false
34.
35. 'A' >= 'a'
36. -- true
37.
38. SELECT title, author_lname FROM books WHERE author_lname = 'Eggers';
39.
40. SELECT title, author_lname FROM books WHERE author_lname = 'egggers';
41.
42. SELECT title, author_lname FROM books WHERE author_lname = 'eGGers';

```

LESS THAN

```

1. SELECT title, released_year FROM books;
2.
3. SELECT title, released_year FROM books
4. WHERE released_year < 2000;
5.
6. SELECT title, released_year FROM books
7. WHERE released_year <= 2000;
8.
9. SELECT 3 < -10;
10. -- false
11.
12. SELECT -10 < -9;
13. -- true
14.
15. SELECT 42 <= 42;
16. -- true
17.
18. SELECT 'h' < 'p';
19. -- true
20.
21. SELECT 'Q' <= 'q';
22. -- true

```

LOGICAL AND

```

1. SELECT title, author_lname, released_year FROM books
2. WHERE author_lname='Eggers';
3.
4. SELECT title, author_lname, released_year FROM books
5. WHERE released_year > 2010;
6.
7. SELECT
8.     title,
9.     author_lname,
10.    released_year FROM books
11. WHERE author_lname='Eggers'
12.    AND released_year > 2010;
13.
14. SELECT 1 < 5 && 7 = 9;

```

```

15. -- false
16.
17. SELECT -10 > -20 && 0 <= 0;
18. -- true
19.
20. SELECT -40 <= 0 AND 10 > 40;
21. --false
22.
23. SELECT 54 <= 54 && 'a' = 'A';
24. -- true
25.
26. SELECT *
27. FROM books
28. WHERE author_lname='Eggers'
29.       AND released_year > 2010
30.       AND title LIKE '%novel%';

```

LOGICAL OR

```

1. SELECT
2.     title,
3.     author_lname,
4.     released_year
5. FROM books
6. WHERE author_lname='Eggers' || released_year > 2010;
7.
8.
9. SELECT 40 <= 100 || -2 > 0;
10. -- true
11.
12. SELECT 10 > 5 || 5 = 5;
13. -- true
14.
15. SELECT 'a' = 5 || 3000 > 2000;
16. -- true
17.
18. SELECT title,
19.         author_lname,
20.         released_year,
21.         stock_quantity
22. FROM   books
23. WHERE  author_lname = 'Eggers'
24.        || released_year > 2010
25. OR     stock_quantity > 100;

```

BETWEEN

```

1. SELECT title, released_year FROM books WHERE released_year >= 2004 &&
   released_year <= 2015;
2.
3. SELECT title, released_year FROM books
4. WHERE released_year BETWEEN 2004 AND 2015;
5.
6. SELECT title, released_year FROM books
7. WHERE released_year NOT BETWEEN 2004 AND 2015;
8.
9. SELECT CAST('2017-05-02' AS DATETIME);
10.
11. show databases;
12.
13. use new_testing_db;
14.
15. SELECT name, birthdt FROM people WHERE birthdt BETWEEN '1980-01-01' AND
   '2000-01-01';
16.
17. SELECT
18.     name,

```

```

19.     birthdt
20. FROM people
21. WHERE
22.     birthdt BETWEEN CAST('1980-01-01' AS DATETIME)
23.     AND CAST('2000-01-01' AS DATETIME);

```

IN AND NOT IN

```

1. show databases();
2. use book_shop;
3.
4. SELECT
5.     title,
6.     author_lname
7. FROM books
8. WHERE author_lname='Carver' OR
9.        author_lname='Lahiri' OR
10.       author_lname='Smith';
11.
12. SELECT title, author_lname FROM books
13. WHERE author_lname IN ('Carver', 'Lahiri', 'Smith');
14.
15. SELECT title, released_year FROM books
16. WHERE released_year IN (2017, 1985);
17.
18. SELECT title, released_year FROM books
19. WHERE released_year != 2000 AND
20.        released_year != 2002 AND
21.        released_year != 2004 AND
22.        released_year != 2006 AND
23.        released_year != 2008 AND
24.        released_year != 2010 AND
25.        released_year != 2012 AND
26.        released_year != 2014 AND
27.        released_year != 2016;
28.
29. SELECT title, released_year FROM books
30. WHERE released_year NOT IN
31. (2000,2002,2004,2006,2008,2010,2012,2014,2016);
32.
33. SELECT title, released_year FROM books
34. WHERE released_year >= 2000
35. AND released_year NOT IN
36. (2000,2002,2004,2006,2008,2010,2012,2014,2016);
37.
38. SELECT title, released_year FROM books
39. WHERE released_year >= 2000 AND
40. released_year % 2 != 0;
41.
42. SELECT title, released_year FROM books
43. WHERE released_year >= 2000 AND
44. released_year % 2 != 0 ORDER BY released_year;

```

CASE STATEMENTS

```

1. SELECT title, released_year,
2.     CASE
3.         WHEN released_year >= 2000 THEN 'Modern Lit'
4.         ELSE '20th Century Lit'
5.     END AS GENRE
6. FROM books;
7.
8. SELECT title, stock_quantity,
9.     CASE
10.        WHEN stock_quantity BETWEEN 0 AND 50 THEN '*'
11.        WHEN stock_quantity BETWEEN 51 AND 100 THEN '**'
12.        ELSE '***'

```

```

13.     END AS STOCK
14. FROM books;
15.
16. SELECT title,
17.     CASE
18.         WHEN stock_quantity BETWEEN 0 AND 50 THEN '*'
19.         WHEN stock_quantity BETWEEN 51 AND 100 THEN '**'
20.         ELSE '***'
21.     END AS STOCK
22. FROM books;
23.
24. SELECT title, stock_quantity,
25.     CASE
26.         WHEN stock_quantity BETWEEN 0 AND 50 THEN '*'
27.         WHEN stock_quantity BETWEEN 51 AND 100 THEN '**'
28.         WHEN stock_quantity BETWEEN 101 AND 150 THEN '***'
29.         ELSE '****'
30.     END AS STOCK
31. FROM books;
32.
33. SELECT title, stock_quantity,
34.     CASE
35.         WHEN stock_quantity <= 50 THEN '*'
36.         WHEN stock_quantity <= 100 THEN '**'
37.         ELSE '***'
38.     END
39. AS STOCK
40. FROM books;

```

LOGICAL OPERATORS EXERCISES

```

1. SELECT 10 != 10;
2. -- false
3.
4. SELECT 15 > 14 && 99 - 5 <= 94;
5. -- true
6.
7. SELECT 1 IN (5,3) || 9 BETWEEN 8 AND 10;
8. -- true
9.
10. SELECT title, released_year FROM books WHERE released_year < 1980;
11.
12. SELECT title, author_lname FROM books WHERE author_lname='Eggers' OR
    author_lname='Chabon';
13.
14. SELECT title, author_lname FROM books WHERE author_lname IN
    ('Eggers','Chabon');
15.
16. SELECT title, author_lname, released_year FROM books WHERE author_lname
    = 'Lahiri' && released_year > 2000;
17.
18. SELECT title, pages FROM books WHERE pages >= 100 && pages <=200;
19.
20. SELECT title, pages FROM books WHERE pages BETWEEN 100 AND 200;
21.
22. SELECT
23.     title,
24.     author_lname
25. FROM books
26. WHERE
27.     author_lname LIKE 'C%' OR
28.     author_lname LIKE 'S%';
29.
30. SELECT
31.     title,
32.     author_lname
33. FROM books
34. WHERE

```

```

35.     SUBSTR(author_lname,1,1) = 'C' OR
36.     SUBSTR(author_lname,1,1) = 'S';
37.
38. SELECT title, author_lname FROM books
39. WHERE SUBSTR(author_lname,1,1) IN ('C', 'S');
40.
41. SELECT
42.     title,
43.     author_lname,
44.     CASE
45.         WHEN title LIKE '%stories%' THEN 'Short Stories'
46.         WHEN title = 'Just Kids' OR title = 'A Heartbreaking Work of
Staggering Genius' THEN 'Memoir'
47.         ELSE 'Novel'
48.     END AS TYPE
49. FROM books;
50.
51. SELECT author_fname, author_lname,
52.     CASE
53.         WHEN COUNT(*) = 1 THEN '1 book'
54.         ELSE CONCAT(COUNT(*), ' books')
55.     END AS COUNT
56. FROM books
57. GROUP BY author_lname, author_fname;

```

ONE TO MANY

WORKING WITH FOREIGN KEYS

-- Creating the customers and orders tables

```

1. CREATE TABLE customers(
2.     id INT AUTO_INCREMENT PRIMARY KEY,
3.     first_name VARCHAR(100),
4.     last_name VARCHAR(100),
5.     email VARCHAR(100)
6. );
7. CREATE TABLE orders(
8.     id INT AUTO_INCREMENT PRIMARY KEY,
9.     order_date DATE,
10.    amount DECIMAL(8,2),
11.    customer_id INT,
12.    FOREIGN KEY(customer_id) REFERENCES customers(id)
13. );

```

-- Inserting some customers and orders

```

1. INSERT INTO customers (first_name, last_name, email)
2. VALUES ('Boy', 'George', 'george@gmail.com'),
3.     ('George', 'Michael', 'gm@gmail.com'),
4.     ('David', 'Bowie', 'david@gmail.com'),
5.     ('Blue', 'Steele', 'blue@gmail.com'),
6.     ('Bette', 'Davis', 'bette@aol.com');
7.
8. INSERT INTO orders (order_date, amount, customer_id)
9. VALUES ('2016/02/10', 99.99, 1),
10.     ('2017/11/11', 35.50, 1),
11.     ('2014/12/12', 800.67, 2),
12.     ('2015/01/03', 12.50, 2),
13.     ('1999/04/11', 450.25, 5);

```

-- This INSERT fails because of our fk constraint. No user with id: 98

```
1. INSERT INTO orders (order_date, amount, customer_id)
2. VALUES ('2016/06/06', 33.67, 98);
```

CROSS JOINS

-- Finding Orders Placed By George: 2 Step Process

```
3. SELECT id FROM customers WHERE last_name='George';
4. SELECT * FROM orders WHERE customer_id = 1;
```

-- Finding Orders Placed By George: Using a subquery

```
1. SELECT * FROM orders WHERE customer_id =
2.     (
3.         SELECT id FROM customers
4.         WHERE last_name='George'
5.     );
```

-- Cross Join Crazyiness

```
SELECT * FROM customers, orders;
```

INNER JOINS

-- IMPLICIT INNER JOIN

```
1. SELECT * FROM customers, orders
2. WHERE customers.id = orders.customer_id;
```

-- IMPLICIT INNER JOIN

```
1. SELECT first_name, last_name, order_date, amount
2. FROM customers, orders
3.     WHERE customers.id = orders.customer_id;
```

-- EXPLICIT INNER JOINS

```
1. SELECT * FROM customers
2. JOIN orders
3.     ON customers.id = orders.customer_id;
4.
5. SELECT first_name, last_name, order_date, amount
6. FROM customers
7. JOIN orders
8.     ON customers.id = orders.customer_id;
9.
10. SELECT *
11. FROM orders
12. JOIN customers
13.     ON customers.id = orders.customer_id;
```

-- ARBITRARY JOIN - meaningless, but still possible

```
1. SELECT * FROM customers
2. JOIN orders ON customers.id = orders.id;
```

```
SET @@SQL_MODE=";
```

#Expression #1 of SELECT list is not in GROUP BY clause and contains nonaggregated column 'bookshop.books.title' which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by

LEFT JOINS

-- Getting Fancier (Inner Joins Still)

```
1. SELECT first_name, last_name, order_date, amount
2. FROM customers
3. JOIN orders
4.     ON customers.id = orders.customer_id
5. ORDER BY order_date;
```

```
1. SELECT
2.     first_name,
3.     last_name,
4.     SUM(amount) AS total_spent
5. FROM customers
6. JOIN orders
7.     ON customers.id = orders.customer_id
8. GROUP BY orders.customer_id
9. ORDER BY total_spent DESC;
```

-- LEFT JOINS

```
1. SELECT * FROM customers
2. LEFT JOIN orders
3.     ON customers.id = orders.customer_id;
```

```
1. SELECT first_name, last_name, order_date, amount
2. FROM customers
3. LEFT JOIN orders
4.     ON customers.id = orders.customer_id;
```

```
1. SELECT
2.     first_name,
3.     last_name,
4.     IFNULL(SUM(amount), 0) AS total_spent
5. FROM customers
6. LEFT JOIN orders
7.     ON customers.id = orders.customer_id
8. GROUP BY customers.id
9. ORDER BY total_spent;
```

RIGHT JOINS

-- OUR FIRST RIGHT JOIN (seems the same as a left join?)

```
SELECT * FROM customers
RIGHT JOIN orders
    ON customers.id = orders.customer_id;
```

-- ALTERING OUR SCHEMA to allow for a better example (optional)

```
1. CREATE TABLE customers(
2.     id INT AUTO_INCREMENT PRIMARY KEY,
3.     first_name VARCHAR(100),
4.     last_name VARCHAR(100),
5.     email VARCHAR(100)
6. );
7. CREATE TABLE orders(
```

```

8.      id INT AUTO_INCREMENT PRIMARY KEY,
9.      order_date DATE,
10.     amount DECIMAL(8,2),
11.     customer_id INT
12. );

```

-- INSERTING NEW DATA (no longer bound by foreign key constraint)

```

1. INSERT INTO customers (first_name, last_name, email)
2. VALUES ('Boy', 'George', 'george@gmail.com'),
3.          ('George', 'Michael', 'gm@gmail.com'),
4.          ('David', 'Bowie', 'david@gmail.com'),
5.          ('Blue', 'Steele', 'blue@gmail.com'),
6.          ('Bette', 'Davis', 'bette@aol.com');
7.
8. INSERT INTO orders (order_date, amount, customer_id)
9. VALUES ('2016/02/10', 99.99, 1),
10.        ('2017/11/11', 35.50, 1),
11.        ('2014/12/12', 800.67, 2),
12.        ('2015/01/03', 12.50, 2),
13.        ('1999/04/11', 450.25, 5);
14.
15. INSERT INTO orders (order_date, amount, customer_id) VALUES
16. ('2017/11/05', 23.45, 45),
17. (CURDATE(), 777.77, 109);

```

--A MORE COMPLEX RIGHT JOIN

```

1. SELECT
2.     IFNULL(first_name, 'MISSING') AS first,
3.     IFNULL(last_name, 'USER') as last,
4.     order_date,
5.     amount,
6.     SUM(amount)
7. FROM customers
8. RIGHT JOIN orders
9.     ON customers.id = orders.customer_id
10. GROUP BY first_name, last_name;

```

-- WORKING WITH ON DELETE CASCADE

```

1. CREATE TABLE customers(
2.     id INT AUTO_INCREMENT PRIMARY KEY,
3.     first_name VARCHAR(100),
4.     last_name VARCHAR(100),
5.     email VARCHAR(100)
6. );
7.
8. CREATE TABLE orders(
9.     id INT AUTO_INCREMENT PRIMARY KEY,
10.    order_date DATE,
11.    amount DECIMAL(8,2),
12.    customer_id INT,
13.    FOREIGN KEY(customer_id)
14.        REFERENCES customers(id)
15.        ON DELETE CASCADE
16. );
17.
18.
19. INSERT INTO customers (first_name, last_name, email)
20. VALUES ('Boy', 'George', 'george@gmail.com'),
21.          ('George', 'Michael', 'gm@gmail.com'),
22.          ('David', 'Bowie', 'david@gmail.com'),
23.          ('Blue', 'Steele', 'blue@gmail.com'),

```

```

24.         ('Bette', 'Davis', 'bette@aol.com');
25.
26. INSERT INTO orders (order_date, amount, customer_id)
27. VALUES ('2016/02/10', 99.99, 1),
28.         ('2017/11/11', 35.50, 1),
29.         ('2014/12/12', 800.67, 2),
30.         ('2015/01/03', 12.50, 2),
31.         ('1999/04/11', 450.25, 5);

```

RIGHT AND LEFT JOINS FAQ

```

1. SELECT * FROM customers
2. LEFT JOIN orders
3.     ON customers.id = orders.customer_id;

```

```

1. SELECT * FROM orders
2. RIGHT JOIN customers
3.     ON customers.id = orders.customer_id;

```

```

1. SELECT * FROM orders
2. LEFT JOIN customers
3.     ON customers.id = orders.customer_id;

```

```

1. SELECT * FROM customers
2. RIGHT JOIN orders
3.     ON customers.id = orders.customer_id;

```

JOINS EXERCISE

-- The Schema

```

1. CREATE TABLE students (
2.     id INT AUTO_INCREMENT PRIMARY KEY,
3.     first_name VARCHAR(100)
4. );
5.
6.
7. CREATE TABLE papers (
8.     title VARCHAR(100),
9.     grade INT,
10.    student_id INT,
11.    FOREIGN KEY (student_id)
12.        REFERENCES students(id)
13.        ON DELETE CASCADE
14. );

```

-- The Starter Data

```

1. INSERT INTO students (first_name) VALUES
2. ('Caleb'),
3. ('Samantha'),
4. ('Raj'),
5. ('Carlos'),
6. ('Lisa');
7.
8. INSERT INTO papers (student_id, title, grade ) VALUES
9. (1, 'My First Book Report', 60),
10. (1, 'My Second Book Report', 75),
11. (2, 'Russian Lit Through The Ages', 94),
12. (2, 'De Montaigne and The Art of The Essay', 98),
13. (4, 'Borges and Magical Realism', 89);

```

-- EXERCISE 1

```
1. SELECT first_name, title, grade
2. FROM students
3. INNER JOIN papers
4.     ON students.id = papers.student_id
5. ORDER BY grade DESC;
```

-- ALT SOLUTION

```
1. SELECT first_name, title, grade
2. FROM students
3. RIGHT JOIN papers
4.     ON students.id = papers.student_id
5. ORDER BY grade DESC;
```

-- PROBLEM 2

```
1. SELECT first_name, title, grade
2. FROM students
3. LEFT JOIN papers
4.     ON students.id = papers.student_id;
```

-- PROBLEM 3

```
1. SELECT
2.     first_name,
3.     IFNULL(title, 'MISSING'),
4.     IFNULL(grade, 0)
5. FROM students
6. LEFT JOIN papers
7.     ON students.id = papers.student_id;
```

-- PROBLEM 4

```
1. SELECT
2.     first_name,
3.     IFNULL(AVG(grade), 0) AS average
4. FROM students
5. LEFT JOIN papers
6.     ON students.id = papers.student_id
7. GROUP BY students.id
8. ORDER BY average DESC;
```

-- PROBLEM 5

```
1. SELECT first_name,
2.     Ifnull(Avg(grade), 0) AS average,
3.     CASE
4.         WHEN Avg(grade) IS NULL THEN 'FAILING'
5.         WHEN Avg(grade) >= 75 THEN 'PASSING'
6.         ELSE 'FAILING'
7.     end AS passing_status
8. FROM students
9. LEFT JOIN papers
10.     ON students.id = papers.student_id
11. GROUP BY students.id
12. ORDER BY average DESC;
```

MANY TO MANY

TV REVIEW

-- CREATING THE REVIEWERS TABLE

```
1. CREATE TABLE reviewers (  
2.     id INT AUTO_INCREMENT PRIMARY KEY,  
3.     first_name VARCHAR(100),  
4.     last_name VARCHAR(100)  
5. );
```

-- CREATING THE SERIES TABLE

```
1. CREATE TABLE series(  
2.     id INT AUTO_INCREMENT PRIMARY KEY,  
3.     title VARCHAR(100),  
4.     released_year YEAR(4),  
5.     genre VARCHAR(100)  
6. );
```

-- CREATING THE REVIEWS TABLE

```
1. CREATE TABLE reviews (  
2.     id INT AUTO_INCREMENT PRIMARY KEY,  
3.     rating DECIMAL(2,1),  
4.     series_id INT,  
5.     reviewer_id INT,  
6.     FOREIGN KEY(series_id) REFERENCES series(id),  
7.     FOREIGN KEY(reviewer_id) REFERENCES reviewers(id)  
8. );
```

-- INSERTING A BUNCH OF DATA

```
1. INSERT INTO series (title, released_year, genre) VALUES  
2.     ('Archer', 2009, 'Animation'),  
3.     ('Arrested Development', 2003, 'Comedy'),  
4.     ('Bob's Burgers', 2011, 'Animation'),  
5.     ('Bojack Horseman', 2014, 'Animation'),  
6.     ('Breaking Bad', 2008, 'Drama'),  
7.     ('Curb Your Enthusiasm', 2000, 'Comedy'),  
8.     ('Fargo', 2014, 'Drama'),  
9.     ('Freaks and Geeks', 1999, 'Comedy'),  
10.    ('General Hospital', 1963, 'Drama'),  
11.    ('Halt and Catch Fire', 2014, 'Drama'),  
12.    ('Malcolm In The Middle', 2000, 'Comedy'),  
13.    ('Pushing Daisies', 2007, 'Comedy'),  
14.    ('Seinfeld', 1989, 'Comedy'),  
15.    ('Stranger Things', 2016, 'Drama');  
16.  
17.  
18. INSERT INTO reviewers (first_name, last_name) VALUES  
19.     ('Thomas', 'Stoneman'),  
20.     ('Wyatt', 'Skaggs'),  
21.     ('Kimbra', 'Masters'),  
22.     ('Domingo', 'Cortes'),  
23.     ('Colt', 'Steele'),  
24.     ('Pinkie', 'Petit'),  
25.     ('Marlon', 'Crafford');  
26.  
27.  
28. INSERT INTO reviews(series_id, reviewer_id, rating) VALUES  
29.     (1,1,8.0), (1,2,7.5), (1,3,8.5), (1,4,7.7), (1,5,8.9),  
30.     (2,1,8.1), (2,4,6.0), (2,3,8.0), (2,6,8.4), (2,5,9.9),  
31.     (3,1,7.0), (3,6,7.5), (3,4,8.0), (3,3,7.1), (3,5,8.0),  
32.     (4,1,7.5), (4,3,7.8), (4,4,8.3), (4,2,7.6), (4,5,8.5),  
33.     (5,1,9.5), (5,3,9.0), (5,4,9.1), (5,2,9.3), (5,5,9.9),  
34.     (6,2,6.5), (6,3,7.8), (6,4,8.8), (6,2,8.4), (6,5,9.1),
```

```

35.      (7,2,9.1),(7,5,9.7),
36.      (8,4,8.5),(8,2,7.8),(8,6,8.8),(8,5,9.3),
37.      (9,2,5.5),(9,3,6.8),(9,4,5.8),(9,6,4.3),(9,5,4.5),
38.      (10,5,9.9),
39.      (13,3,8.0),(13,4,7.2),
40.      (14,2,8.5),(14,3,8.9),(14,4,8.9);

```

Joins Challenge 1

```

1. SELECT
2.     title,
3.     rating
4. FROM series
5. JOIN reviews
6.     ON series.id = reviews.series_id;

```

Challenge 2 AVG rating

```

1. SELECT
2.     title,
3.     AVG(rating) as avg_rating
4. FROM series
5. JOIN reviews
6.     ON series.id = reviews.series_id
7. GROUP BY series.id
8. ORDER BY avg_rating;

```

CHALLENGE 3 - Two Solutions

```

1. SELECT
2.     first_name,
3.     last_name,
4.     rating
5. FROM reviewers
6. INNER JOIN reviews
7.     ON reviewers.id = reviews.reviewer_id;

```

```

1. SELECT
2.     first_name,
3.     last_name,
4.     rating
5. FROM reviews
6. INNER JOIN reviewers
7.     ON reviewers.id = reviews.reviewer_id;

```

CHALLENGE 4 - UNREVIEWED SERIES

```

1. SELECT title AS unreviewed_series
2. FROM series
3. LEFT JOIN reviews
4.     ON series.id = reviews.series_id
5. WHERE rating IS NULL;

```

Challenge 5 - GENRE AVG RATINGS

```

1. SELECT genre,
2.     Round(Avg(rating), 2) AS avg_rating
3. FROM series
4. INNER JOIN reviews
5.     ON series.id = reviews.series_id
6. GROUP BY genre;

```

HALLENGE 6 - Reviewer Stats

```
1. SELECT first_name,
2.         last_name,
3.         Count(rating) AS COUNT,
4.         Ifnull(Min(rating), 0) AS MIN,
5.         Ifnull(Max(rating), 0) AS MAX,
6.         Round(Ifnull(Avg(rating), 0), 2) AS AVG,
7.         IF(Count(rating) > 0, 'ACTIVE', 'INACTIVE') AS STATUS
8. FROM   reviewers
9.       LEFT JOIN reviews
10.      ON reviewers.id = reviews.reviewer_id
11. GROUP BY reviewers.id;
```

CHALLENGE 6 - Reviewer Stats With POWER USERS

```
1. SELECT first_name,
2.         last_name,
3.         Count(rating) AS COUNT,
4.         Ifnull(Min(rating), 0) AS MIN,
5.         Ifnull(Max(rating), 0) AS MAX,
6.         Round(Ifnull(Avg(rating), 0), 2) AS AVG,
7.         CASE
8.             WHEN Count(rating) >= 10 THEN 'POWER USER'
9.             WHEN Count(rating) > 0 THEN 'ACTIVE'
10.            ELSE 'INACTIVE'
11.         end AS STATUS
12. FROM   reviewers
13.       LEFT JOIN reviews
14.      ON reviewers.id = reviews.reviewer_id
15. GROUP BY reviewers.id;
```

CHALLENGE 7 - 3 TABLES!

```
1. SELECT
2.     title,
3.     rating,
4.     CONCAT(first_name, ' ', last_name) AS reviewer
5. FROM reviewers
6. INNER JOIN reviews
7.     ON reviewers.id = reviews.reviewer_id
8. INNER JOIN series
9.     ON series.id = reviews.series_id
10. ORDER BY title;
```

IG CLONE

USERS SCHEMA

```
1. CREATE TABLE users (
2.     id INTEGER AUTO_INCREMENT PRIMARY KEY,
3.     username VARCHAR(255) UNIQUE NOT NULL,
4.     created_at TIMESTAMP DEFAULT NOW()
5. );
```

PHOTOS SCHEMA

```
1. CREATE TABLE photos (
2.     id INTEGER AUTO_INCREMENT PRIMARY KEY,
3.     image_url VARCHAR(255) NOT NULL,
4.     user_id INTEGER NOT NULL,
5.     created_at TIMESTAMP DEFAULT NOW(),
6.     FOREIGN KEY(user_id) REFERENCES users(id)
7. );
```

COMMENTS SCHEMA

```
1. CREATE TABLE comments (  
2.     id INTEGER AUTO_INCREMENT PRIMARY KEY,  
3.     comment_text VARCHAR(255) NOT NULL,  
4.     photo_id INTEGER NOT NULL,  
5.     user_id INTEGER NOT NULL,  
6.     created_at TIMESTAMP DEFAULT NOW(),  
7.     FOREIGN KEY(photo_id) REFERENCES photos(id),  
8.     FOREIGN KEY(user_id) REFERENCES users(id)  
9. );
```

LIKES SCHEMA

```
1. CREATE TABLE likes (  
2.     user_id INTEGER NOT NULL,  
3.     photo_id INTEGER NOT NULL,  
4.     created_at TIMESTAMP DEFAULT NOW(),  
5.     FOREIGN KEY(user_id) REFERENCES users(id),  
6.     FOREIGN KEY(photo_id) REFERENCES photos(id),  
7.     PRIMARY KEY(user_id, photo_id)  
8. );
```

FOLLOWERS SCHEMA

```
1. CREATE TABLE follows (  
2.     follower_id INTEGER NOT NULL,  
3.     followee_id INTEGER NOT NULL,  
4.     created_at TIMESTAMP DEFAULT NOW(),  
5.     FOREIGN KEY(follower_id) REFERENCES users(id),  
6.     FOREIGN KEY(followee_id) REFERENCES users(id),  
7.     PRIMARY KEY(follower_id, followee_id)  
8. );
```

HASHTAGS SCHEMA

```
1. CREATE TABLE tags (  
2.     id INTEGER AUTO_INCREMENT PRIMARY KEY,  
3.     tag_name VARCHAR(255) UNIQUE,  
4.     created_at TIMESTAMP DEFAULT NOW()  
5. );  
6.  
7. CREATE TABLE photo_tags (  
8.     photo_id INTEGER NOT NULL,  
9.     tag_id INTEGER NOT NULL,  
10.    FOREIGN KEY(photo_id) REFERENCES photos(id),  
11.    FOREIGN KEY(tag_id) REFERENCES tags(id),  
12.    PRIMARY KEY(photo_id, tag_id)  
13. );
```

SECTION 15: WORKING WITH LOTS OF INSTAGRAM DATA

Dataset:

<https://www.udemy.com/course/the-ultimate-mysql-bootcamp-go-from-sql-beginner-to-expert/learn/lecture/7026326#overview>

-- 1. FINDING 5 OLDEST USERS

```
1. SELECT *  
2. FROM users  
3. ORDER BY created_at  
4. LIMIT 5;
```

-- 2. MOST POPULAR REGISTRATION DATE

```
1. SELECT
2.     DAYNAME(created_at) AS day,
3.     COUNT(*) AS total
4. FROM users
5. GROUP BY day
6. ORDER BY total DESC
7. LIMIT 2;
```

-- 3. IDENTIFY INACTIVE USERS (USERS WITH NO PHOTOS)

```
1. SELECT username
2. FROM users
3. LEFT JOIN photos
4.     ON users.id = photos.user_id
5. WHERE photos.id IS NULL;
```

-- 4. IDENTIFY MOST POPULAR PHOTO (AND USER WHO CREATED IT)

```
1. SELECT
2.     username,
3.     photos.id,
4.     photos.image_url,
5.     COUNT(*) AS total
6. FROM photos
7. INNER JOIN likes
8.     ON likes.photo_id = photos.id
9. INNER JOIN users
10.    ON photos.user_id = users.id
11. GROUP BY photos.id
12. ORDER BY total DESC
13. LIMIT 1;
```

-- 5. CALCULATE AVERAGE NUMBER OF PHOTOS PER USER

```
1. SELECT (SELECT Count(*)
2.         FROM photos) / (SELECT Count(*)
3.         FROM users) AS avg;
```

-- 6. FIND THE FIVE MOST POPULAR HASHTAGS

```
SELECT tags.tag_name,
       Count(*) AS total
FROM photo_tags
     JOIN tags
       ON photo_tags.tag_id = tags.id
GROUP BY tags.id
ORDER BY total DESC
LIMIT 5;
```

-- 7. FINDING THE BOTS - THE USERS WHO HAVE LIKED EVERY SINGLE PHOTO

```
1. SELECT username,
2.     Count(*) AS num_likes
3. FROM users
4.     INNER JOIN likes
5.         ON users.id = likes.user_id
6. GROUP BY likes.user_id
7. HAVING num_likes = (SELECT Count(*)
8.                     FROM photos);
```

To calculate the number of bots, take your query and make it a subquery like this:

```
1. SELECT COUNT(*) AS bot_count
2. FROM (
3. SELECT
4.   COUNT(users.id)
5. FROM comments
6. RIGHT JOIN users
7.   ON users.id = comments.user_id
8. GROUP BY users.id
9. HAVING COUNT(*) = (SELECT COUNT(*) FROM photos)
10. ) AS bots;
```

You can combine it all into several different outer queries and subqueries. What I've done below is count and display the total users, the total BOTs, and the total Celebrities and calculated the percentage of BOTs and Celebrities.

```
1. SELECT
2.   COUNT(DISTINCT(users.username)) AS TotalUsers
3. , (SELECT COUNT(*) as TotAllComments FROM
4.   (
5.     SELECT count(username) as NumAll FROM users u1
6.     LEFT JOIN comments on u1.id = comments.user_id
7.     GROUP BY u1.username
8.     HAVING COUNT(comment_text) = (SELECT COUNT(*) FROM photos)
9.   ) b) AS Bots
10. , (SELECT count(users.username) FROM users WHERE users.id NOT
11.   IN(SELECT user_id FROM comments)) as Celebrities
11. , CONCAT(ROUND((SELECT count(users.username) FROM users WHERE
12.   users.id NOT IN(SELECT user_id FROM
13.   comments))/COUNT(DISTINCT(username))*100,0),'%') Celebrity_Percentage
12. , CONCAT(ROUND((SELECT COUNT(*) as TotAllComments FROM
13.   (
14.     SELECT count(username) as NumAll FROM users u1
15.     LEFT JOIN comments on u1.id = comments.user_id
16.     GROUP BY u1.username
17.     HAVING COUNT(comment_text) = (SELECT COUNT(*) FROM
18.   photos)
18.   ) c)/COUNT(DISTINCT(username))*100,0),'%')
19.   Bot_Percentage
19. FROM users
20. LEFT JOIN comments on users.id = comments.user_id
21. ;
```

BULK INSERTING 500 FAKE USERS NODE.JS + MYSQL (PYTHON FAKER)

```
1. var mysql = require('mysql');
2. var faker = require('faker');
3. var connection = mysql.createConnection({
4.   host      : 'localhost',
5.   user      : 'root',
6.   password  : '',
7.   database  : 'join_us'
8. });
9. var data = [];
10. for(var i = 0; i < 500; i++){
11.   a. data.push([
12.     i. faker.internet.email(),
13.     ii. faker.date.past()
14.   b. ]);
11. }
12. var q = 'INSERT INTO users (email, created_at) VALUES ?';
13. connection.query(q, [data], function(err, result) {
```

```

14. console.log(err);
15. console.log(result);
16. });
17. connection.end();

```

500 USERS EXERCISES

CHALLENGE 1. FIND EARLIEST DATE A USER JOINED

```

1. SELECT
2.     DATE_FORMAT(MIN(created_at), "%M %D %Y") as earliest_date
3. FROM users;

```

CHALLENGE 2. FIND EMAIL OF THE FIRST (EARLIEST) USER

```

1. SELECT *
2. FROM   users
3. WHERE  created_at = (SELECT Min(created_at)
4.                      FROM   users);

```

CHALLENGE 3. USERS ACCORDING TO THE MONTH THEY JOINED

```

1. SELECT Monthname(created_at) AS month,
2.        Count(*)              AS count
3. FROM   users
4. GROUP BY month
5. ORDER BY count DESC;

```

CHALLENGE 4. COUNT NUMBER OF USERS WITH YAHOO EMAILS

```

1. SELECT Count(*) AS yahoo_users
2. FROM   users
3. WHERE  email LIKE '%@yahoo.com';

```

CHALLENGE 5. CALCULATE TOTAL NUMBER OF USERS FOR EACH EMAIL HOST

```

1. SELECT CASE
2.     WHEN email LIKE '%@gmail.com' THEN 'gmail'
3.     WHEN email LIKE '%@yahoo.com' THEN 'yahoo'
4.     WHEN email LIKE '%@hotmail.com' THEN 'hotmail'
5.     ELSE 'other'
6. end AS provider,
7. Count(*) AS total_users
8. FROM   users
9. GROUP BY provider
10. ORDER BY total_users DESC;

```

[HTTPS://STACKOVERFLOW.COM/QUESTIONS/2628138/HOW-TO-SELECT-DOMAIN-NAME-FROM-EMAIL-ADDRESS](https://stackoverflow.com/questions/2628138/how-to-select-domain-name-from-email-address)

The inner SUBSTR gets the right part of the email address after @ and the outer SUBSTRING_INDEX will cut off the result at the first period.

```

SELECT
  (SUBSTRING_INDEX
    (SUBSTR(email, INSTR(email, '@') + 1), '.', 1)) AS host,
  count(*) AS num FROM users

```

GROUP BY host;

If domain is expected to contain multiple words like `mail.yahoo.com`

```
SELECT
RIGHT (email, length(email)-INSTR(email, '@'))
AS host_name
FROM users;
```

Middle Step:

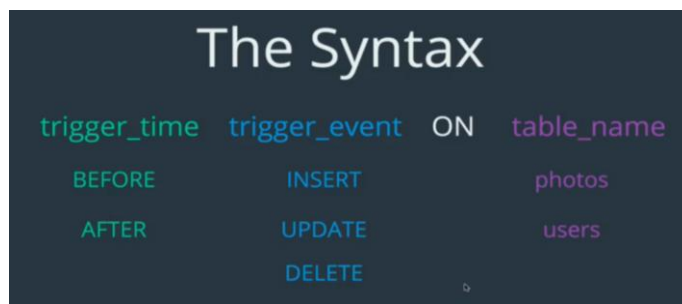
```
SELECT email, INSTR(email, '@') FROM users;
[OUTPUT: nparker@gmail.com 8]
```

MYSQL DATABASE TRIGGERS

SQL statements that are **AUTOMATICALLY RUN** when a specific table is changed

Syntax:

```
CREATE TRIGGER trigger_name
trigger_time trigger_event ON table_name FOR EACH ROW
BEGIN
...
END;
```



DELIMITER \$\$

```
CREATE TRIGGER must_be_adult
BEFORE INSERT ON users FOR EACH ROW
BEGIN
IF NEW.age < 18
THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Must be an adult!';
END IF;
END;
```

\$\$ DELIMITER;

Refers to data that is about to be inserted

A numeric error code (1146). This number is MySQL-specific

A five-character SQLSTATE value ('42S02').

The values are taken from ANSI SQL and ODBC and are more standardized.

A message string - textual description of the error

45000 = A generic state representing "unhandled user-defined exception"

LISTING TRIGGERS

```
SHOW TRIGGERS;
```

REMOVING TRIGGERS

```
DROP TRIGGER trigger_name;
```

A WORD OF WARNING

Triggers can make debugging hard!

PREVENTING SELF-FOLLOWS

```
DELIMITER $$
```

```
CREATE TRIGGER trigger_name
  trigger_time trigger_event ON table_name FOR EACH ROW
  BEGIN
  END;
$$
```

```
DELIMITER ;
```

```
DELIMITER $$
```

```
CREATE TRIGGER example_cannot_follow_self
  BEFORE INSERT ON follows FOR EACH ROW
  BEGIN
    IF NEW.follower_id = NEW.followee_id
    THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Cannot follow yourself';
    END IF;
  END;
$$
```

```
DELIMITER ;
```

LOGGING UNFOLLOWS

```
DELIMITER $$
```

```
CREATE TRIGGER trigger_name
  trigger_time trigger_event ON table_name FOR EACH ROW
  BEGIN
  END;
$$
```

```
DELIMITER ;
```

```
DELIMITER $$
```

```
CREATE TRIGGER create_unfollow
  AFTER DELETE ON follows FOR EACH ROW
BEGIN
  INSERT INTO unfollows
    SET follower_id = OLD.follower_id,
       followee_id = OLD.followee_id;
END$$

DELIMITER ;
```
